

Pratik Dev

Backend Engineer (Go) with a full-stack background — 4+ years of production experience across React/TypeScript and Go. Currently focused on backend engineering: building production-style Go systems covering REST APIs, async job queues, Redis-backed high-throughput serving, and distributed systems patterns. Strong frontend depth makes cross-functional collaboration and API integration second nature.

Personal Information

- **Email:** pratikdevofficial@gmail.com
- **Phone:** +880-1537220785
- **Socials:** [GitHub](#), [Twitter X](#), [LinkedIn](#), [Portfolio](#)
- **Address:** Chattogram, Bangladesh

Education

- **B.Sc. in Computer Science and Engineering**
 - **Institution:** BGC Trust University, Chattogram, Bangladesh
 - **Duration:** 2022-2023
 - **Result:** Dropout

Skills

Technical Skills:

- **Languages:** Golang, TypeScript, JavaScript, Node.js
- **Backend:** pgx/v5, PostgreSQL, Redis, Docker, golang-migrate, slog
- **Frameworks:** React.js, Next.js, Express.js
- **Frontend:** Tailwind CSS, ShadcnUI, Framer Motion, Bootstrap
- **Databases:** PostgreSQL, Redis, MongoDB, MySQL, SQLite
- **Technologies:** Docker, Appwrite, Firebase, Supabase, Convex, Vercel
- **AI Tools:** Codex, Claude Code, Opencode
- **Tools:** Git, GitHub, VS Code, Postman, Linear, Figma

Fundamentals:

- Data Structures, Algorithms, System Design, Concurrency Patterns

Soft Skills:

- Communication, Ownership, Problem-solving, Collaboration, Creativity

Projects

Backend & System Projects

- [Result Lookup](#)
 - **Description:** High-throughput exam result API simulating Bangladesh's SSC like result publishing infrastructure – designed to serve 2M student results at a single fixed publish moment under sustained peak load. Precomputes all results into Redis before T0 as pre-serialized JSON, uses a publish gate flag for atomic release, and falls back to Postgres under Redis failure with connection limiting to prevent cascade failure.
 - **Tech Stack:** Golang, PostgreSQL, Redis, pgx/v5, Docker, golang-migrate, slog
 - **Challenges:** Cursor-based batch seeding (50k rows/batch) with count verification before gate flip; Redis key design for O(1) exact-match lookup with zero JSON marshaling on the hot path; Postgres fallback with `MaxConns` capping to protect the database under Redis failure.
 - **Engineering practices:** Load tested with `hey` – 31,910 RPS peak on a single instance, zero errors across 615,000 total requests; handler tests use `miniredis` for real Redis behavior without a live instance; two-binary architecture (API + precompute job) sharing internal packages.
- [URL Health Checker](#)
 - **Description:** An async job processing system – submit a URL, a background worker claims it via `SELECT ... FOR UPDATE SKIP LOCKED`, performs an HTTP health check, and retries on failure with exponential backoff. Two binaries (API + worker) from one codebase, sharing a PostgreSQL-backed job queue with no external message broker.
 - **Tech Stack:** Golang, PostgreSQL, pgx/v5, Docker, golang-migrate, slog
 - **Challenges:** Designed stale-job recovery directly into the claim query (staleness threshold replacing a heartbeat process), exponential backoff written in SQL to keep retry state atomic with the status update, and graceful worker shutdown via `signal.NotifyContext` so the process never dies mid-database-update.

- **Engineering practices:** Separate Dockerfiles per binary; `PerformHealthCheck` unit-tested with `httptest.NewServer` including timeout simulation at 500ms (not the real 10s); integration tests against a real PostgreSQL instance with schema loaded from actual migration files via `//go:embed`.
 - [URL Shortener API](#) | [Live Demo](#)
 - **Description:** A production-style URL shortener API built with Go and PostgreSQL, with raw SQL (no ORM), versioned migrations, structured logging, and a full unit + integration test suite. Deployed live on Render.
 - **Tech Stack:** Golang, PostgreSQL, pgx/v5, Docker, golang-migrate, slog
 - **Challenges:** Built a per-IP token bucket rate limiter from scratch using a generic thread-safe map, with the refill/consume logic running inside a single mutex-protected operation to eliminate a read-then-write race condition – verified correct under concurrent load using Go's race detector (`-race`), not just manual testing.
 - **Engineering practices:** Multi-stage Docker build running as a non-root user; integration tests run against a real PostgreSQL instance (not mocked) with schema loaded directly from the migration files; retry-on-conflict logic for short code generation instead of pre-checking, with collisions correctly classified as server errors rather than client conflicts.
 - [URL Scraper](#)
 - **Description:** A terminal-based URL scraper that extracts URLs from a given link.
 - **Tech Stack:** Golang
 - **Challenges:** Handling invalid URLs, avoiding duplicate links using data structures, and implementing concurrency with goroutines and mutex locks.
 - [The Super Tiny Compiler](#)
 - **Description:** A minimal compiler built from scratch using Go, implementing parsing logic and tree-based data structures for representing code structure.
 - **Tech Stack:** Golang, Abstract Syntax Trees (AST), Data Structures
-

Full-stack Projects

- [Roadmap App](#)
 - **Description:** A full-stack roadmap application that allows users to view, upvote, filter, sort and comment on roadmap items.
 - **Tech Stack:** Next.js v15, Tailwind CSS, TypeScript, Zod, Drizzle ORM, PostgreSQL, Vercel
 - **Challenges:** Implementing user authentication and managing state effectively.
 - **Source:** <https://github.com/PratikDev/roadmap-app>

- **[Narrative Guard](#)**
 - **Description:** Narrative Guard helps teams audit content against their brand constitution before publishing.
 - **Tech Stack:** Next.js v15, Tailwind CSS, TypeScript, Zod, Convex, Gemini, Vercel
 - **Challenges:** Designing a trustworthy AI/RAG audit flow with explainable scoring, workspace permissions, and reliable brand-specific context retrieval.
 - **Source:** <https://github.com/PratikDev/narrative-guard>
-

Front-end Projects

- **[Hello World Communications](#)**
 - **Description:** A landing page for an IT solutions company.
 - **Tech Stack:** Next.js v15, ShadcnUI, Tailwind CSS, TypeScript, Framer Motion, Zod
 - **Challenges:** Implementing dynamic content and ensuring fast loading times
- **[Kar Communication](#)**
 - **Description:** A landing page for an IT solutions company.
 - **Tech Stack:** Next.js v15, ShadcnUI, Tailwind CSS, TypeScript, Framer Motion, Zod, Firebase, React Query
 - **Challenges:** Implementing dynamic content and ensuring fast loading times.

Professional Experience

- **Freelancer** (2021 - Oct 2023)
 - **Description:** Built and delivered full-stack web applications and APIs for multiple clients, working across frontend, backend, and deployment workflows.
 - **Link:** [Fiverr Profile](#)
- **Full-Stack Engineer** at **Bilsida** (Oct 2023 - Aug 2024)
 - **Description:** Developed the frontend and backend of web apps and APIs for a Swedish car marketplace startup, working directly with founders during the company's pre-investment phase.
 - **Tech Stack:** Next.js, Tailwind CSS, ShadcnUI, TypeScript, Appwrite, Docker
 - **Impact:** Built core product UI and backend integrations from scratch, helping take the product from early development to investor-ready state.
- **Full-Stack Engineer** at **[Hello World Communications](#)** (Sep 2024 - Jul 2025)

- **Description:** Built and maintained frontend applications, backend services, and APIs across multiple concurrent client projects for a software development agency.
- **Tech Stack:** Next.js, Tailwind CSS, ShadcnUI, TypeScript, Drizzle ORM, Appwrite, Firebase, Docker
- **Impact:** Owned engineering delivery across client projects end-to-end – from architecture decisions to deployment – directly responsible for client-facing UI quality and reliability.
- **Frontend Engineer** at [Hone](#) (Aug 2025 - Sep 2025)
 - **Description:** Built and maintained user interfaces for both Web and Desktop applications, focusing on clean UI, performance, and scalable component architecture.
 - **Tech Stack:** ReactJS, Tailwind CSS, ShadcnUI, TypeScript
 - **Impact:** Built reusable, accessible UI components that improved both developer experience and frontend consistency across the codebase.
- **Full-Stack Engineer** at [Osilion](#) (Oct 2025 - Feb 2026)
 - **Description:** Developed and maintained full-stack features for Osilion's intelligent recruitment platform, working closely with the team on technical architecture decisions.
 - **Tech Stack:** Next.js, TypeScript, Azure, PostgreSQL, Tailwind CSS, ShadcnUI
 - **Impact:** Helped define design patterns and component architecture used across multiple product surfaces, balancing speed of delivery with long-term maintainability.
- **Software Engineer** at [Devspace](#) (March 2026 - Present)
 - **Description:** Building AI-powered product features, integrating modern agentic AI workflows (Mastra AI, Vercel AI SDK) into a production React/TypeScript application.
 - **Tech Stack:** React, TypeScript, Mastra AI, Convex, OpenCode, Vercel AI SDK, Gemini
 - **Impact:** Contributing to scalable frontend architecture while shipping AI-driven features across the full product lifecycle.

Additional information available upon request.